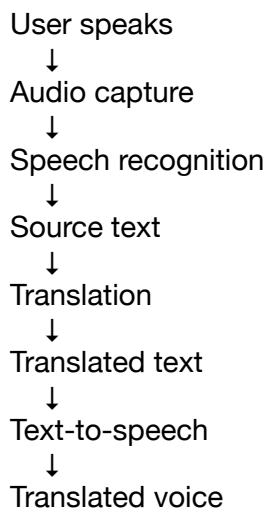


Auto Transliteration mobile first website

What problem the application solves:

A native arabic speaker that doesnt speak English may face difficulty communicating with another foreign person, this software solves this issue by providing a speech to translate to another language.

System flow:



The flow is simply
microphone input -> text (voice to text) -> translate text -> audio (text to speech)

Functional documentation:

- Select source language
- Select target language
- Start/stop translation
- Display recognized speech
- Display translated speech
- Play translated audio

Technical documentation:

- React Native
- Audio capture
- WebSocket/HTTP communication
- Speech-to-text provider
- Translation provider
- TTS provider
- Backend

Decision making:

- React Native was selected because the application requires mobile-native microphone and audio functionality while maintaining a shared Android/iOS codebase.
- Google cloud API was chosen for its proven capabilities and trusted results.

External services:

Azure Cloud:

1- Azure only does STT:

Purpose: translate incoming speech into a text

Input: voice

Output: translated text

Pricing/free tier: 5 hours for each month for free

Failure behavior:

User speaks



Azure



NoMatch

treat it as ("No speech recognized")

— — —

Azure



AuthenticationFailure

treat it as (Speech service authentication failed.)

— — —

Bad request



STOP



Don't retry unchanged request

— — —

Azure quota exceeded



STOP API requests



Do NOT retry



Tell user service limit was reached

Google Cloud:

2- Translation NMT:

Purpose: translate text to a selected language

Input: text

Output: translated text

Pricing/free tier: first 500k per month is free

Failure behavior:

Treat it the same as above

3- Text-to-Speech AI:

Purpose: translate text to a voice

Input: text

Output: voice

Pricing/free tier: Standard 4 million characters for free

Failure behavior:

Treat it the same as above

Data flow:

Phone



Backend



Speech API



Translation API



TTS API



Backend



Phone

The data should be encrypted, temporarily processed, then deleted.

Architecture:

Mobile App



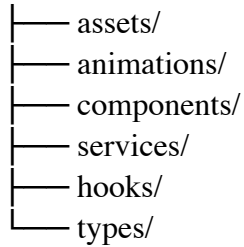
Backend



AI Services

Implementation:

src/



Cost constraints:

- **No unlimited translation sessions.** Every user must have a maximum amount of translation time per day/month.
- **Set an application-level audio limit.** Do not allow a user to leave microphone translation running indefinitely.
- **Automatically stop translation after the configured session limit.**
- **Require the user to explicitly start a new session** after the limit is reached.
- **Do not continuously send silent audio** to Google. Detect silence and stop/pause processing where appropriate.
- **Do not send every tiny partial transcript to Translation.** Translate meaningful finalized segments instead.
- **Do not regenerate TTS unnecessarily.** Only synthesize speech when the translated text has actually changed/finalized.
- **Limit maximum text size per translation request.**
- **Limit concurrent translation sessions per user.**
- **Limit concurrent sessions globally** so a sudden traffic spike cannot create uncontrolled API usage.
- **Reject requests when the application's internal quota has been reached.**
- **Track usage per user and globally.**
- **Maintain a server-side usage counter** rather than trusting the mobile application to report its own usage.
- **Set Google Cloud budget alerts** so you are notified when spending approaches your defined budget.
- **Use Google Cloud quotas in addition to your own limits.**
- **Never expose Google Cloud service-account credentials in the mobile application.**

Design:

